

RL-Acrobot

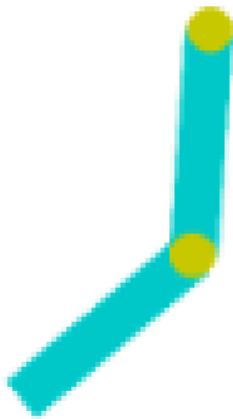
2025-11-16

Reinforcement Learning project with gymnasiums Acrobot

Katja Suckow

Einleitung Acrobot

Das System besteht aus 2 Ketten und 2 Verbindungen. Die untere Kette schwingt frei. Das Ziel ist es, durch Bewegungen der Ketten einen Drehmoment zu erzeugen, um von der hängenden Ausgangsposition die zweite freie Kette über eine vorgegebene Höhe zu schwingen.



Einleitung workbench und configurator

Workbench

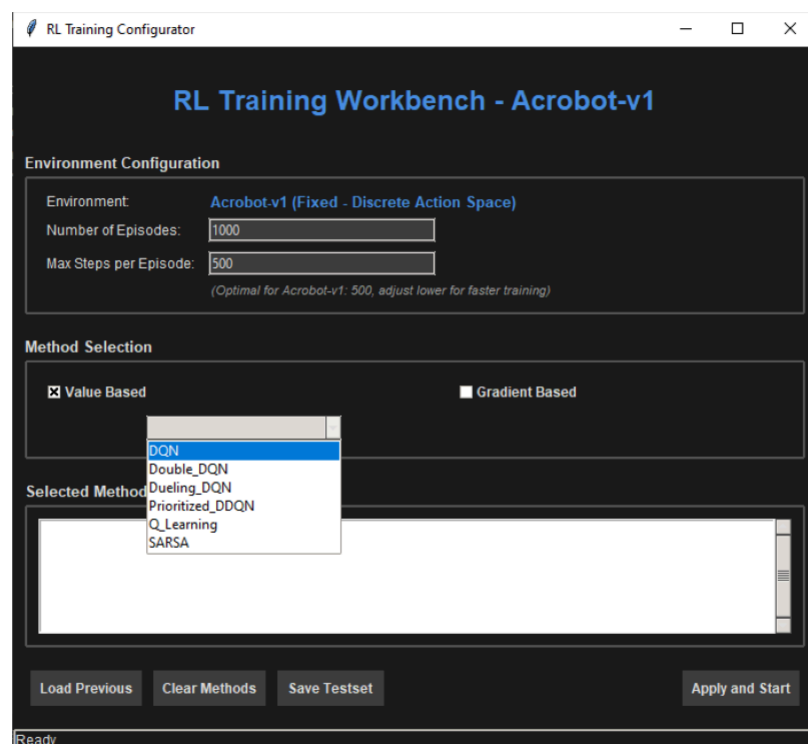
- als 1. Schritt wurde eine Workbench eingerichtet, damit ein Konfigurator von einem Runner getrennt implementiert werden kann
- bei der Implementierung der Workbench wurden die Requirements für die Implementierung des Projektes festgehalten

1. **Environment** Classic Control Gymnasium Umgebung: **Acrobot**

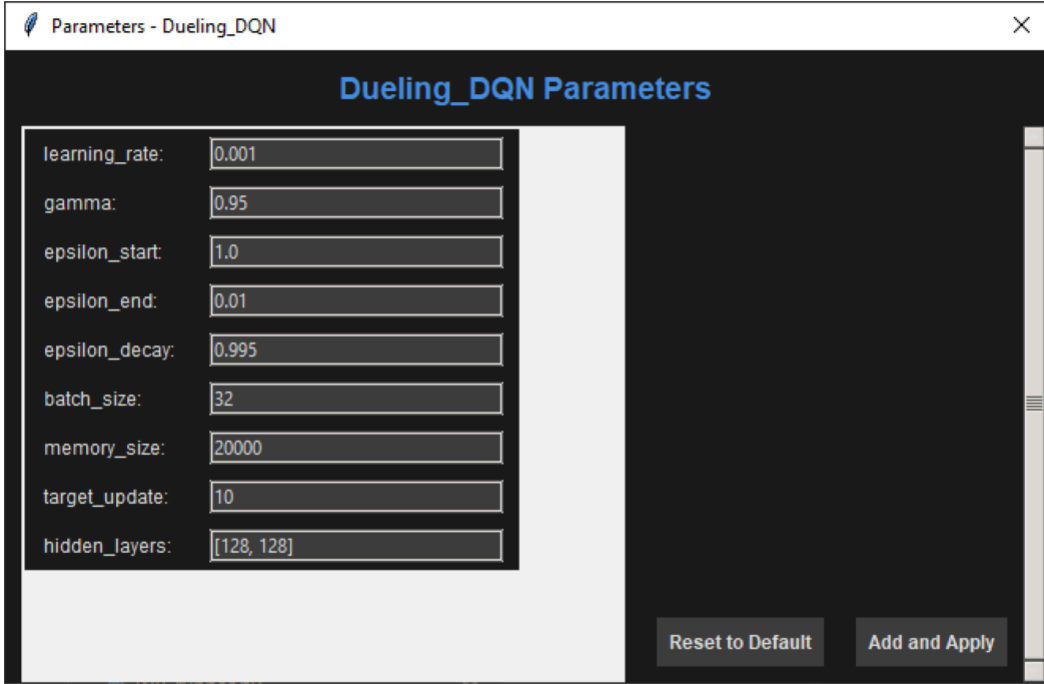
2. **Agent** Trainingsmethoden in der Environment, Algorithmen zur Beschreibung wie sich Agent durch Environment bewegt: **Double DQN, Prioritized DQN, SARSA**
3. **Visualisierungstools** welche Metriken sollen den Lernverlauf visuell/grafisch darstellen: **Reward** and **MAV**
4. **Hyperparameter** welche Hyperparameter werden zu einer möglichen Lernoptimierung eingestellt: **default, freigestellt**

Konfigurator

- In dem Kontext des Projektes soll der Konfigurator einen Ablauf definieren, bei der Nutzer im Interface Einstellungen für das Training vornehmen können, diese Einstellungen speichert und dann an den Runner für das Training übergibt.
1. Es wird eine GUI geöffnet, in der das Environment “Acrobot” schon hardcoded vorgegeben ist.
 2. Einige Parametereinstellungen (Episodenanzahl, Anzahl der Schritte pro Episode) können hier verändert werden. Diese werden dann für alle Methoden beim Training gleich sein.
 3. Darunter stehen die verschiedenen Typen der Trainingsalgorithmen zur Auswahl
 4. Methode lässt sich in einem Dropdown Menü auswählen.



4. öffnet sich ein neues Fenster, bei der die Parameter für die zugehörige vom Nutzer eingestellt und verändert werden können. Nutzer können die Werte auch wieder auf Default zurücksetzen.
5. Wenn die Parameter für die Methode gewählt sind, drücken Nutzer auf “Add and Apply” und die Werte werden übernommen und das Pop-up Window schliesst sich.

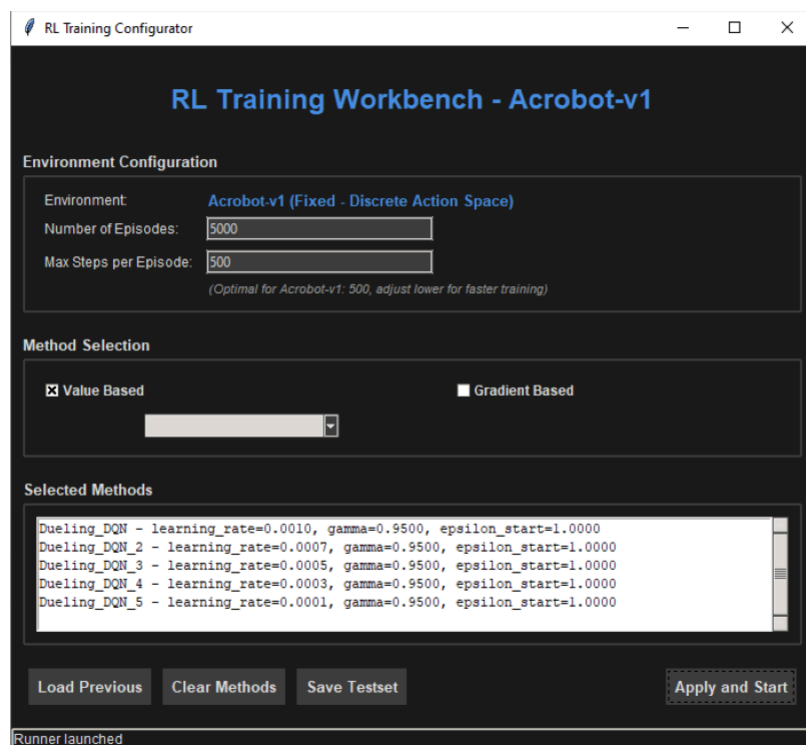
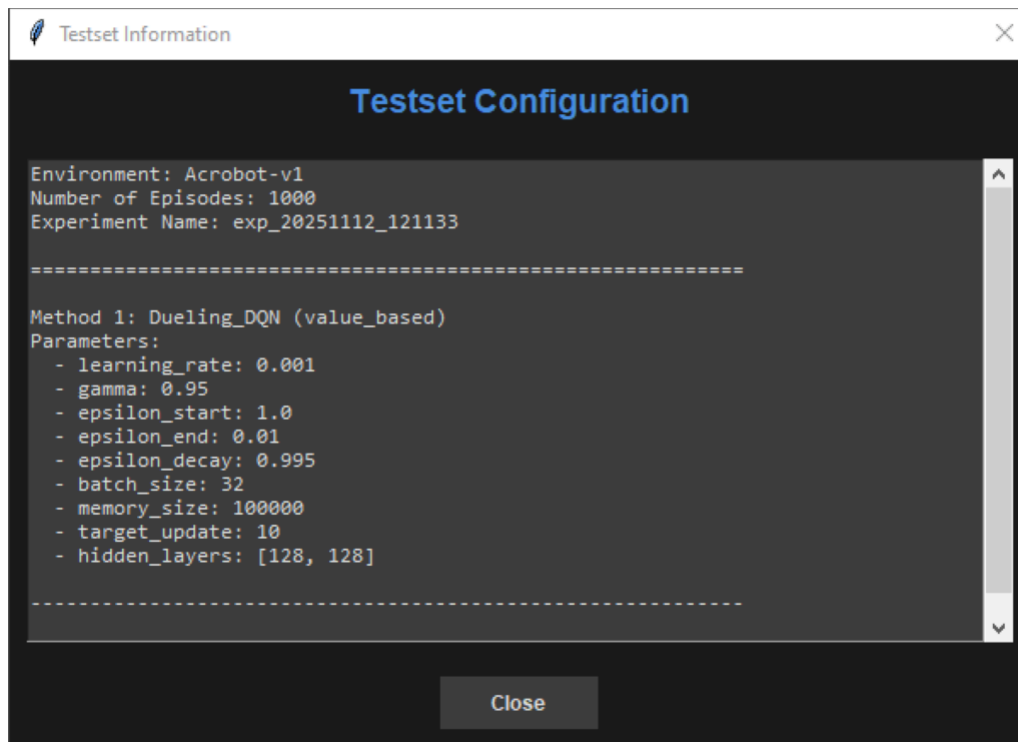


The screenshot shows a window titled "Parameters - Dueling_DQN" with a close button (X) in the top right corner. The main title "Dueling_DQN Parameters" is centered at the top. Below the title, there is a list of parameters with their corresponding values in input fields:

Parameter	Value
learning_rate:	0.001
gamma:	0.95
epsilon_start:	1.0
epsilon_end:	0.01
epsilon_decay:	0.995
batch_size:	32
memory_size:	20000
target_update:	10
hidden_layers:	[128, 128]

At the bottom right of the window, there are two buttons: "Reset to Default" and "Add and Apply".

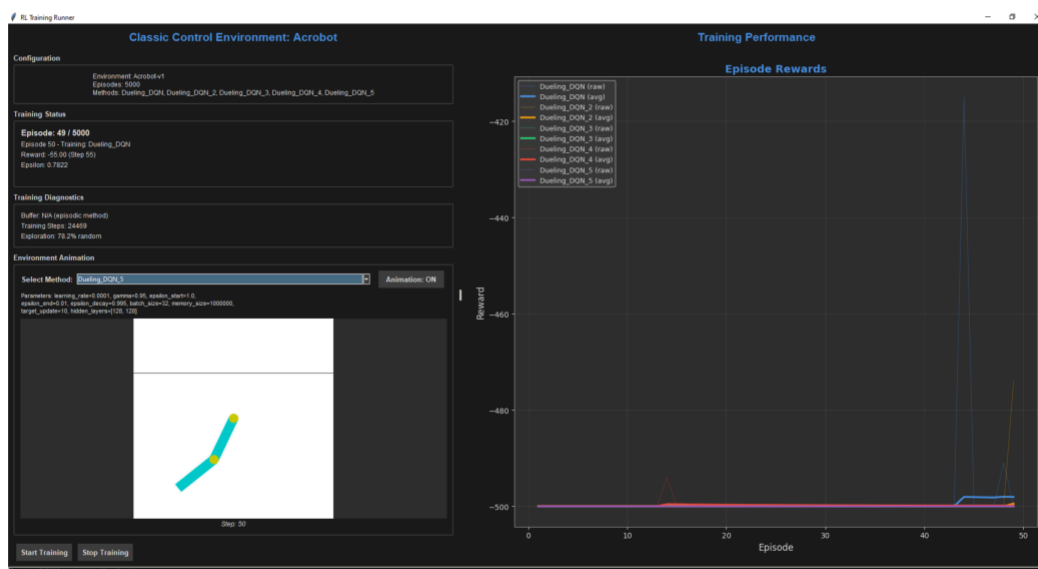
6. Im Hauptfenster werden die Werte in einer Liste dargestellt. Nutzer haben die Möglichkeit eine weitere Methode zu wählen und dann die Parameter wie vorher beschrieben auszuwählen. Diese werden anschliessend in der dargestellten List angehängt.
7. Man kann diese Liste als "Testset" in einem json File speichern, man kann auch schon vorher gespeicherte Testsets noch einmal laden.
8. Sollte sich der Nutzer dafür entscheiden, bereits gespeicherte Testsets noch einmal zu laden, erscheint ein Pop-up Window und der Nutzer kann diese Werte noch einmal einsehen.
9. Mit "Add und Start" werden die Werte an den Runner übergeben.



Runner

Der Runner übernimmt die Werte des Testsets vom Konfigurator und nutzt diese für sein Training

- Konfigurator übergibt dem Runner die Werte des Testsets als Dictionary Variable direkt
- Testset mit den verschiedenen gespeicherten Trainingsmethoden und deren Parametern werden im Projekt für die Acrobot Environment durchgeführt
- Es öffnet sich ein neues großes Fenster: in einem Bereich wird per matplotlib der Trainingsverlauf aller Methoden in einem real-life Plot zeigt.
 - X-Achse ein Wert für jede durchlaufene Episode, Y-Achse ist der Reward
- Zusätzlich wird in einem weiteren Fenster noch die ausgewählte Animation Gymnasiums “Acrobot”, die mit den Schritten der jeweiligen Episode synchronisiert ist
- Man kann die Anzeige der Animation über einen Toggle ein- und ausschalten: zusätzlich lässt sich einstellen, welche der parallel laufenden Trainingsmethoden animiert angezeigt werden soll.
- Ansonsten werden im Runner Fenster noch Informationen zur Konfiguration, zum Trainingsstatus (aktuelle Episode, Schrittverlauf pro Episode)
- Ein “Start Training” Button beginnt das Training
 - aktiviert Training, startet plotting und updated Plot nach jeder Episode (einzelne Schritte einer Episode kann man in der Animation beobachten)
 - das Training der verschiedenen Methoden erfolgt parallel. Obwohl dies den Trainingsprozess verlangsamt lässt sich der Prozess und Unterschied zwischen den Methoden während des Trainings beobachten und etwas früher einschätzen.
- Ein “Stop Training” Button
 - Training wird beendet
 - Rewartergebnisse jeder Episode über alle parallel trainierten Einträge des Testsets werden in einer .json Datei gespeichert in einem Unterordner des result Ordners gespeichert.



Introduction to the different Learning Methods and their Parameters

Double DQN

Eine Variante des DQN, das versucht overestimation (Q-Werte von einigen erfolgreichen Trainingsstep werden überschätzt) zu vermeiden oder verringern. Zwei Netzwerke werden gebaut:

- **Online Netzwerk** selektiert Aktionen von einem aktuellen Zustand aus.
 - wählt die Aktion mit den höchsten Q-Werten für den nächsten Zustand aus
- **Target Netzwerk** schätzt die Q-values der selektierten Aktionen
 - berechnet den Q-Wert für die Aktion, um die Overestimation zu vermeiden

Prioritized DQN

Erfahrungen werden priorisiert: basiert auf temporal difference Fehler. Ermittelt den Unterschied zwischen vorhergesagtem und aktuellem Q-Wert. Die mit höheren TD Fehlern werden häufiger angewendet.

SARSA

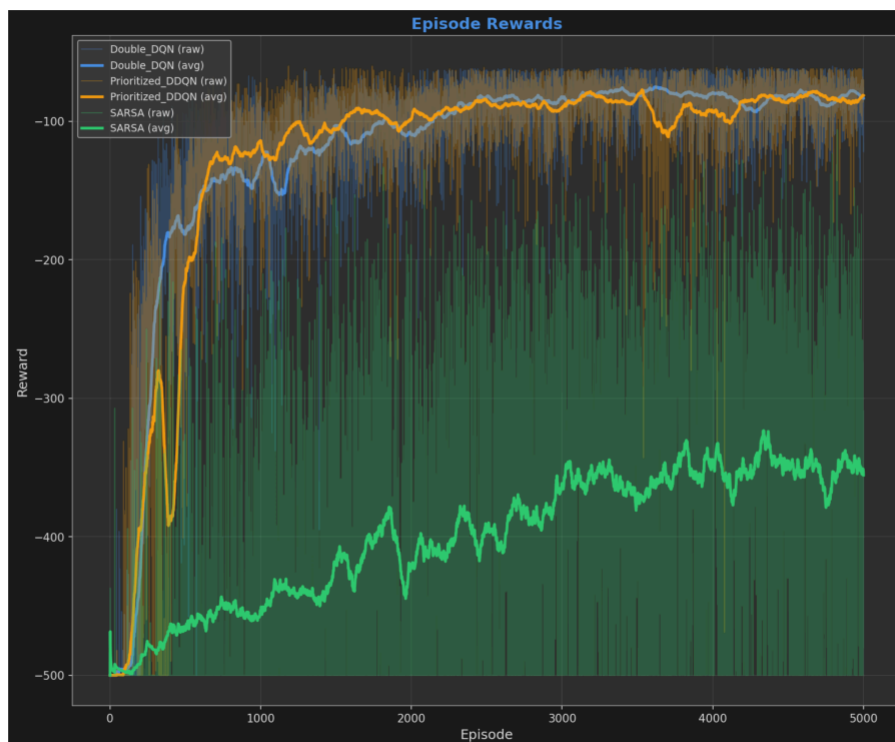
lernt auf Basis der vergangener Aktion eine Action-Value Funktion nach gerade verwendeter Policy. * wendet dabei aktuell gelernte Policy, Werte werden nach der Erfahrung jeweils geschätzt, während die gerade gelernte Policy angewendet wird.

Resultate

Drei Methoden gleichzeitig mit ihren Default Parametern 5000 Episoden laufen lassen - 500 Schritte für jede Episode

- jeweils mit ihren Default Parametern getestet.

Plot below: Direkter Vergleich: Double-DQN, Prioritized-DQN, SARSA



Auswertung des Reinforcement Learning Trainings

- Jedem der Schritte in einer Episode wird ein Return Wert von Maximal 0 gegeben. Der Reward Wert ergibt sich aus der Summe der Returns der einzelnen Schritte
- im obigen Training wurde für jede Episode die Summe der einzelnen Returns zu einem Reward Wert summiert. Der Rewardwert kann dadurch nur zwischen -500 und 0 liegen
- Der Reward Wert für alle 3 Methoden liegt am Anfang bei -500 und steigt dann im Laufe des Trainings nach oben (unterschiedlich schnell).

Konvergenz, Variabilität, Maximum

Double DQN

- die Lernkurve geht nach ca 100 Episoden steil nach oben
- nach ca 400 Episoden flacht die Kurve etwas ab, aber steigt generell stetig weiter
- dabei variieren die Reward Werte eigentlich kaum
- der maximale Trainingswert liegt bei über -100, ca -50

Prioritized DQN

- wie bei DDQN geht die anfängliche Lernkurve nach dem Start erst steil nach oben
- nach ca 400 Episoden bricht sie aber ein und zeigt mehr Variabilität an dieser Stelle
- Sie erholt sich aber nach ca 500 Episoden wieder, steigt steil und hat DDQN bei ca 600 Episoden aufgeholt
- danach flacht die Lernkurve ab, steigt aber trotzdem stabil, mit wenig Variabilität weiter
- finales Trainingsergebnis nach 5000 Episoden etwa gleich zu DDQN
- Maximaler Reward Wert ist auch bei PDQN zum Schluss ca -50.

SARSA

- SARSA zeigt die ersten Lernergebnisse schon sehr früh nach dem Start des Trainings
- Die Lernkurve bleibt allerdings sehr flach, geht aber stetig nach oben
- die Lernkurve zeigt durchgängig mehr Variabilität als bei den beiden DQN Lernmethoden
- Die jeweiligen Ausschläge sind jedoch recht gering
- der Maximale Reward Wert liegt bei ca -330, also deutlich unter den Maximalwerten von den beiden DQN Algorithmen

Dueling DQN Parameter: Learning Rate mit 1000 Episode

- ist Reinforcement Learning Methode, die im Vergleich zu traditioneller DQN die Value in einem bestimmten Zustand schätzt und den Vorteil wenn man sich für eine bestimmte Aktion in diesem State entscheidet

Value Funktion * Wie gut ist es im aktuellen Zustand zu sein

$$V(s)$$

Advantage Funktion * berechnet Vorteil jeder Aktion in einem gegeben Zustand

$$A(s, a)$$

Diese beiden Funktionen werden kombiniert um Q-Werte zu bestimmen:

$$Q(s, a) = V(s) + (A(s, a) - \max_a A(s, a))$$

Dadurch kann zwischen dem Wert in einem bestimmten Zustand und den Vorteilen einer bestimmten Aktion in diesem Zustand unterschieden werden. Value decomposition soll Stabilität bringen.

Dueling DQN Parameter: Vergleich mit unterschiedlicher Lernrate

Lernrate

- aus Recherche und Erfahrung während des Kurses über die wichtigen Parameter einen weiteren Test die verschiedenen Lernraten miteinander verglichen (bei 1.000.000 Memory Space)
- eine höhere Lernrate kann schnelles Lernen ermöglichen, (Ergebnisse nach weniger Episoden) macht das Ergebnis allerdings weniger stabil, sodass die Rewards (auch MAV) sehr stark variieren können
- eine niedrigere Lernrate lernt später, führt aber über eine längere Trainingsphase zu stabilen Ergebnissen.
- Der Vergleich von 5 verschiedenen Lernraten gleichzeitig hatte eine lange Trainingszeit zur Folge. Deshalb wurden nur 1000 Episoden trainiert.

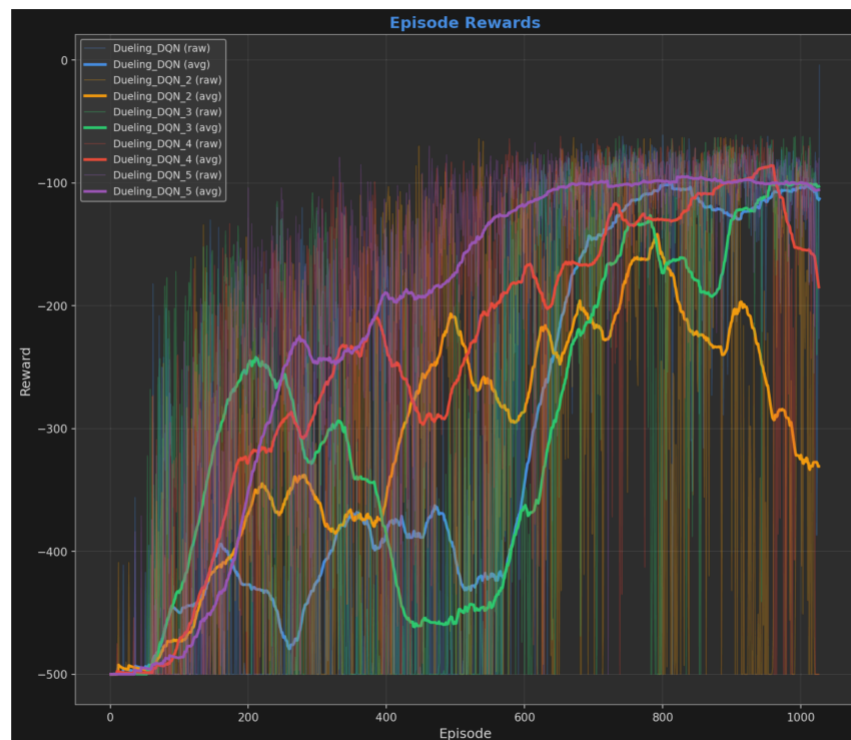
Dueling_DQN learning rate: 0.001

Dueling_DQN_2 learning rate: 0.0007

Dueling_DQN_3 learning rate: 0.0005

Dueling_DQN_4 learning rate: 0.0003

Dueling_DQN_5 learning rate: 0.0001



Beschreibung Parametervergleich Learning Rate in Acrobot

Hier wurden der Effekt 5 unterschiedlicher Lernraten im range 0.001 - 0.0001 mit einander verglichen.

1. 0.001 (blau):

- die Lernrate geht zuerst steil nach oben, bricht aber immer wieder ein
 - zeigt hohe Variabilität
 - höchster Wert: -100
2. 0.0007 (orange):
- Lernrate zuerst steil, bricht dann immer wieder ein und geht dann auch wieder nach unten
 - hohe Variabilität, aber über gesamten Trainingsverlauf etwas besser als blau
 - höchster Wert -160
3. 0.0005 (grün):
- erst steil nach oben, dann großer Einbruch, bis es dann doch wieder hoch geht
 - zeigt bis auf den großen Einbruch keine große Variabilität - steigt gegen Ende noch weiter an
 - höchster Wert: -100
4. 0.0003 (rot):
- steiler Anstieg, dann flacht ab, stetig ansteigend bis zum Schluß
 - Schon auch eine gewisse Variabilität, aber keine großen Einbrüche
 - höchster Wert: ca -90
5. 0.0001 (violett):
- etwas späterer Anstieg im Vergleich zu den anderen Bedingungen, bleibt aber bis zum Schluss stabil ohne Einbrüche
 - kaum Variabilität während des gesamten Trainings
 - höchster Wert: -98

Die Lernraten haben einen Effekt auf das Training, wo bei die kleinste Lernrate 0.0001 die stabilsten und auch die höchsten Reward Werte im Training zeigen. Zwar fängt das Training bei der kleinsten Lernrate etwas später an, aber das ist nicht überraschend.

Größere Lernraten starten mit den Rewardanstiegen zwar früher, fluktuieren während des Trainings sehr häufig und zeigen keine hohen Rewardwerte nach einer ausreichend langen Trainingszeit.

Kleine weitere Besonderheit dieses Trainings

Für Lernmethoden in Acrobot ist eine große Memory Size von Vorteil, damit der Agent beim Lernen auf eine möglichst große Anzahl von vergangenen und vielfältigen Erfahrungen zurückgreifen kann und nicht nur kurzfristige. Deshalb habe ich bei für alle Learning Rate Parameter eine hohe Memory Size von 1.000.000 eingestellt, um möglichst gute Rewardergebnisse beim Training zu bekommen. * für komplexe Environments * hat einen größeren Speicher um sich mehr (zeitlich und divers) merken zu können * nützlich bei komplexeren Environments mit verschiedenen Datentypen * dadurch kann der Algorithmus beim Lernen auf einen größeren Teil seiner bisher gemachten Erfahrungen zurückgreifen

Unterschiedliche Parameter bei Memory Size in Acrobot Umgebung

Internetrecherche schlug bei komplexer Acrobot Umgebung eine Memory Size von 1.000.000 vor. In diesem Projekt wurde bei der Memory Size eine Range von 100.000 bis 1.000.000 getestet.

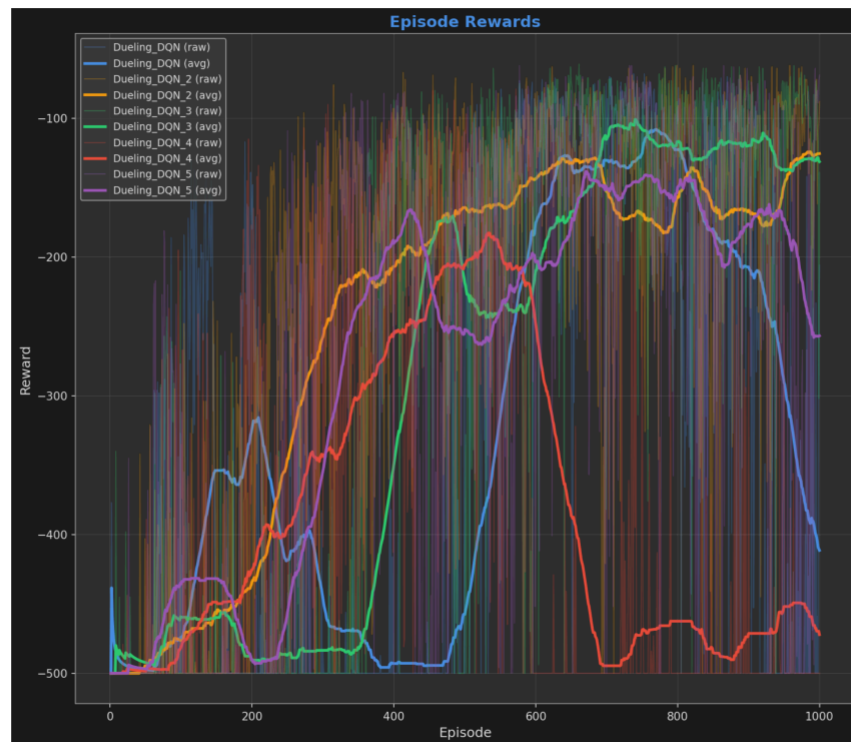
Dueling_DQN - memory size 100.000

Dueling_DQN_2 - memory size 300.000

Dueling_DQN_3 - memory size 500.000

Dueling_DQN_4 - memory size 700.000

Dueling_DQN_5 - memory size 1.000.000



Auswertung Parametervergleich Memory Size

Klare Unterschiede im Trainingsverlauf zwischen den verschiedenen Parametern zu erkennen mit unterschiedlichen Varianzen. Allerdings kann man anhand dieser Grafik nicht mit Konfidenz behaupten, dass sich die Ergebnisse mit jeder größeren memory size stetig verbessern würden. Bei einer Memory Size von 100.000 (blaue Linie) zeigt sich tatsächlich eine große Variabilität im Trainingsverlauf. Im Vergleich zeigt die größte Memory Size von 1.000.000 tatsächlich einen besseren Trainingverlauf mit weniger Variabilität in der Lernphase. Hier muss allerdings der Trainingsverlauf von Dueling DQN bei einer Memory Size von 700.000 besonders betrachtet werden, weil dieser viel schlechter abschneidet, die RewardWerte stürzen nach ca 2500 Episoden komplett ab und erholen sich während des 1000 Episodentrainings nicht mehr.

Trotzdem zeigt die Grafik, dass Memory Size sollte bei der Wahl der Parameter in komplexen Umgebungen wie die Acrobot Umgebung berücksichtigt werden. Die besten und konsistentesten Ergebnisse werden in diesem Episodensample bei einer Memory Size von 300.000 (geringste Variabilität) und 500.000 (höchster Rewardwert ca -100) erzielt.

Wahrscheinlich werden beim Lernprozess unterschiedlich große Memory Sizes (Zugriff auf mehr Erfahrungen im vergangen Lernprozess) mal als nützlich oder weniger nützlich. Ich vermute bei weiteren Episoden wird eine große Memory einen günstigen Einfluss auf die nächsten Entscheidungen haben. Da sich die Werte bis zum Ende der 1000 Episoden nicht stabilisieren, sollte hier noch einmal nachgebessert werden und 5000 Episoden trainiert werden. (Leider ist das Training sehr langsam.)

Abschliessende Bemerkungen und Zusammenfassung

Der Vergleich der 3 Methoden zeigt deutliche Lerneffekte, wobei besonders die beiden DQN Methoden mit ihren Neuronalen Netzen eindeutig bessere Ergebnisse erzielen als die SARSA Methode. Da SARSA stetig steigt, bleibt es offen, ob SARSA die anderen beiden Deep Netze nach einer weiteren langen Trainingszeit noch einholen kann. Die Rewardzeiten am Ende des Trainings scheinen sich stabilisiert zu haben, eigentlich sind die Rewards von über -100 bei einer Schrittzahl von 500 sehr gute Ergebnisse.

Der Vergleich zwischen den Lernraten im Dueling DQN zeigt den vorhergesagten Effekt, dass eine geringere Lernrate bei dieser Methode stabileres Training ermöglicht. Um einen besseren Vergleich zu erzielen, sollten die Trainingsepisoden doch noch auf 5000 verlängert werden. Da fehlte leider die Zeit.

Da die Memory Size im Verlauf des Trainings auch unterschiedliche Effekte zeigte, wäre ein weiterer Vergleich der Lernraten bei einer Memory Size von 300.000 noch interessant. Kann sich das Ergebnis bei einer geringen Lernrate von 300.000 noch weiter verbessern? Beim Vergleich der Memory Size konnten wir beobachten, dass 300.000 Memory Size den stabilsten Trainingsverlauf hatte.

Einige abschliessende Gedanken zum Handling des Konfigurators: Vielleicht wäre noch eine Möglichkeit einzelne Einträge eines bereits erstellten Testsets zu verändern hilfreich im Handling des Konfigurators. Man hat ein Testset erstellt oder ein früheres Testset noch einmal geladen. Momentan kann man zwar alle gemachten Einträge löschen, aber vielleicht möchte der Nutzer doch noch einzelne Einträge korrigieren ohne wie bisher das Testset komplett neu erstellen zu müssen.